JavaScript: Control Statements

Outline

- Selection Structures
  - Single, double, and multiple selection mechanisms
- Repetition Structures
  - Counter-controlled vs. sentinel-controlled repetition
- Operators and Expressions
- DOM Interaction Examples
  - Using document.getElementById
  - Dynamic content update with innerHTML
  - Template literals for string interpolation

7.4 Control Statements (Cont.)

- JavaScript provides three selection structures.
  - The `if` statement either performs (selects) an action if a condition is true or skips the action if the condition is false.
    - Called a single-selection statement because it selects or ignores a single action or group of actions.
  - The `if…else` statement performs an action if a condition is true and performs a different action if the condition is false.
    - Double-selection statement because it selects between two different actions or group of actions.
  - The `switch` statement performs one of many different actions, depending on the value of an expression.
    - Multiple-selection statement because it selects among many different actions or groups of actions.

7.4 Control Statements (Cont.)

- JavaScript provides four repetition statements, namely, `while`, `do…while`, `for` and `for…in`.
- In addition to keywords, JavaScript has other words that are reserved for use by the language, such as the values `null`, `true` and `false`, and words that are reserved for possible future use.

7.6 `if...else` Selection Statement

- Allows you to specify that different actions should be performed when the condition is true and when the condition is false.

7.6 `if...else` Selection Statement (Cont.)

- Conditional operator (`?:`)
  - Closely related to the `if…else` statement
  - JavaScript's only ternary operator—it takes three operands
  - The operands together with the `?:` operator form a conditional expression
  - The first operand is a boolean expression
  - The second is the value for the conditional expression if the boolean expression evaluates to `true`
  - Third is the value for the conditional expression if the boolean expression evaluates to `false`

7.6 if...else Selection Statement (Cont.)

- Nested if…else statements
  - Test for multiple cases by placing if…else statements inside other if…else statements
- The JavaScript interpreter always associates an else with the previous if, unless told to do otherwise by the placement of braces ({})
- The if selection statement expects only one statement in its body
  - To include several statements, enclose the statements in braces ({ and })
  - A set of statements contained within a pair of braces is called a block

7.7 while Repetition Statement

- `while`
  - Allows you to specify that an action is to be repeated while some condition remains true
  - The body of a loop may be a single statement or a block
  - Eventually, the condition becomes false and repetition terminates

7.8 Formulating Algorithms: Counter-Controlled Repetition

- Counter-controlled repetition
  - Often called definite repetition, because the number of repetitions is known before the loop begins executing
- A *total* is a variable in which a script accumulates the sum of a series of values
  - Variables that store totals should normally be initialized to zero before they are used in a script
- A *counter* is a variable a script uses to count—typically in a repetition statement

```
Set total to zero
Set grade counter to one

While grade counter is less than or equal to ten
    Input the next grade
    Add the grade into the total
    Add one to the grade counter

Set the class average to the total divided by ten
Print the class average
```

Fig. 7.6 | Pseudocode algorithm that uses counter-controlled repetition to solve the class-average problem.

```
<!DOCTYPE html>

<!-- Fig. 7.7: average.html -->
<!-- Counter-controlled repetition to calculate a class average. -->
<html>
   <head>
      <meta charset = "utf-8">
      <title>Class Average Program</title>
      <script>

         var total; // sum of grades
         var gradeCounter; // number of grades entered
         var grade; // grade typed by user (as a string)
         var gradeValue; // grade value (converted to integer)
         var average; // average of all grades

         // initialization phase
         total = 0; // clear total
         gradeCounter = 1; // prepare to loop

```

Fig. 7.7 | Counter-controlled repetition to calculate a class average. (Part 1 of 4.)

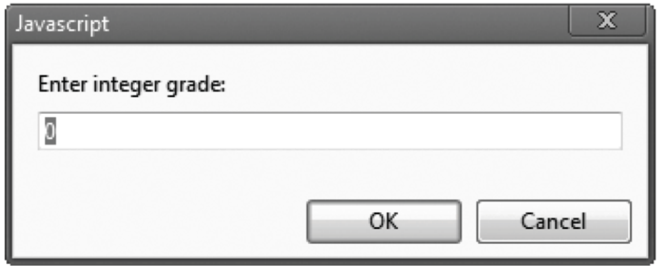
```
      // processing phase
      while ( gradeCounter <= 10 ) // loop 10 times
      {

         // prompt for input and read grade from user
         grade = window.prompt( "Enter integer grade:", "0" );

         // convert grade from a string to an integer
         gradeValue = parseInt( grade );

         // add gradeValue to total
         total = total + gradeValue;

         // add 1 to gradeCounter
         gradeCounter = gradeCounter + 1;
      } // end while

```

Fig. 7.7 | Counter-controlled repetition to calculate a class average. (Part 2 of 4.)

```
            // termination phase
            average = total / 10;   // calculate the average

            // display average of exam grades
            document.writeln(
               "<h1>Class average is " + average + "</h1>" );

         </script>
      </head><body></body>
   </html>
```

Fig. 7.7 | Counter-controlled repetition to calculate a class average. (Part 3 of 4.)

a) This dialog is displayed 10 times. User input is 100, 88, 93, 55, 68, 77, 83, 95, 73 and 62. User enters each grade and presses **OK**.



b) The class average is displayed in a web page

Fig. 7.7 | Counter-controlled repetition to calculate a class average. (Part 4 of 4.)

Not initializing a variable that will be used in a calculation results in a logic error that produces the value NaN—Not a Number. You must initialize the variable before it is used in a calculation.

7.8 Formulating Algorithms: Counter-Controlled Repetition

- JavaScript represents all numbers as floating-point numbers in memory
- Floating-point numbers often develop through division
- The computer allocates only a fixed amount of space to hold such a value, so the stored floating-point value can only be an approximation

If the string passed to `parseInt` contains a floating-point numeric value, `parseInt` simply truncates the floating-point part.

For example, the string "27.95" results in the integer 27, and the string "-123.45" results in the integer -123.

If the string passed to `parseInt` is not a numeric value, `parseInt` returns `NaN` (not a number).

How about “123abcde”?

7.9 Formulating Algorithms: Sentinel-Controlled Repetition

- Sentinel-controlled repetition
  - Special value called a sentinel value (also called a signal value, a dummy value or a flag value) indicates the end of data entry
  - Often is called indefinite repetition, because the number of repetitions is not known in advance
- Choose a sentinel value that cannot be confused with an acceptable input value

```
<!DOCTYPE html>

<!-- Fig. 7.9: average2.html -->
<!-- Sentinel-controlled repetition to calculate a class average. -->
<html>
   <head>
      <meta charset = "utf-8">
      <title>Class Average Program: Sentinel-controlled Repetition</title>
      <script>

         var total; // sum of grades
         var gradeCounter; // number of grades entered
         var grade; // grade typed by user (as a string)
         var gradeValue; // grade value (converted to integer)
         var average; // average of all grades

         // initialization phase
         total = 0; // clear total
         gradeCounter = 0; // prepare to loop

```

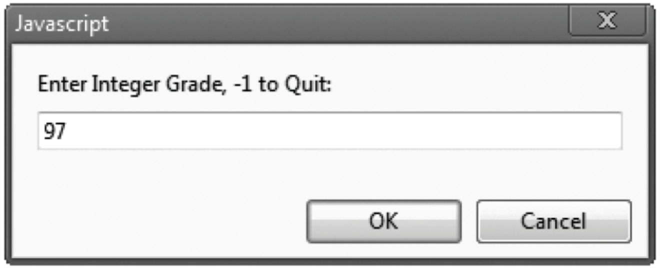
Fig. 7.9 | Sentinel-controlled repetition to calculate a class average. (Part 1 of 4.)

```
        // processing phase
        // prompt for input and read grade from user
        grade = window.prompt(
             "Enter Integer Grade, -1 to Quit:", "0" );

        // convert grade from a string to an integer
        gradeValue = parseInt( grade );

        while ( gradeValue != -1 )
        {
           // add gradeValue to total
           total = total + gradeValue;

           // add 1 to gradeCounter
           gradeCounter = gradeCounter + 1;

           // prompt for input and read grade from user
           grade = window.prompt(
                "Enter Integer Grade, -1 to Quit:", "0" );

           // convert grade from a string to an integer
           gradeValue = parseInt( grade );
        } // end while
```

Fig. 7.9 | Sentinel-controlled repetition to calculate a class average. (Part 2 of 4.)

```

         // termination phase
         if ( gradeCounter != 0 )
         {
            average = total / gradeCounter;

            // display average of exam grades
            document.writeln(
               "<h1>Class average is " + average + "</h1>" );
         } // end if
         else
            document.writeln( "<p>No grades were entered</p>" );

      </script>
   </head><body></body>
</html>
```

Fig. 7.9 | Sentinel-controlled repetition to calculate a class average. (Part 3 of 4.)



This dialog is displayed four times.
User input is 97, 88, 72 and –1.

Fig. 7.9 | Sentinel-controlled repetition to calculate a class average. (Part 4 of 4.)

7.11 Assignment Operators

- JavaScript provides the arithmetic assignment operators +=, -=, *=, /= and %=, which abbreviate certain common types of expressions.

7.12 Increment and Decrement Operators

- The increment operator, ++, and the decrement operator, --, increment or decrement a variable by 1, respectively.
- If the operator is prefixed to the variable, the variable is incremented or decremented by 1, then used in its expression.
- If the operator is postfixed to the variable, the variable is used in its expression, then incremented or decremented by 1.

Operator	Associativity	Type
++ --	right to left	unary
* / %	left to right	multiplicative
+ -	left to right	additive
< <= > >=	left to right	relational
== != === !===	left to right	equality
?:	right to left	conditional
= += -= *= /= %=	right to left	assignment

Fig. 7.15 | Precedence and associativity of the operators discussed so far.

8.2 Essentials of Counter-Controlled Repetition (Cont.)

- The double-quote character delimits the beginning and end of a string literal in JavaScript
  - it cannot be used in a string unless it is preceded by a \ to create the escape sequence \”

8.2 Essentials of Counter-Controlled Repetition (Cont.)

- HTML5 allows either single quotes (') or double quotes (") to be placed around the value of an attribute.
- JavaScript string literals can be enclosed in single quotes ('), double quotes ("), or backticks (`) for template literals.

8.3 for Repetition Statement

- If the loop's condition uses a < or > instead of a <= or >=, or vice-versa, it can result in an off-by-one error
- for statement header contains three expressions
  - Initialization, Condition, and Increment Expression
- The increment expression in the for statement acts like a stand-alone statement at the end of the body of the for statement
- Place only expressions involving the control variable in the initialization and increment sections of a for statement

```
<!DOCTYPE html>

<!-- Fig. 8.2: ForCounter.html -->
<!-- Counter-controlled repetition with the for statement. -->
<html>
   <head>
      <meta charset="utf-8">
      <title>Counter-Controlled Repetition</title>
      <script>

         // Initialization, repetition condition and
         // incrementing are all included in the for
         // statement header.
         for ( var counter = 1; counter <= 7; ++counter )
            document.writeln( "<p style = 'font-size: " +
               counter + "ex'>HTML5 font size " + counter + "ex</p>" );

      </script>
   </head><body></body>
</html>
```

Fig. 8.2 | Counter-controlled repetition with the for statement.

`for` keyword

Control variable *name*

Final value of control variable for which the condition is true

```
for ( var counter = 1; counter <= 7; ++counter )
```

Initial value of control variable

Loop-continuation condition

Increment of control variable

Fig. 8.3 | `for` statement header components.

8.3 for Repetition Statement (Cont.)

- The three expressions in the for statement are optional
- The two semicolons in the for statement are required
- The initialization, loop-continuation condition and increment portions of a for statement can contain arithmetic expressions

```
<!DOCTYPE html>

<!-- Fig. 8.6: Interest.html -->
<!-- Compound interest calculation with a for loop. -->
<html>
   <head>
      <meta charset = "utf-8">
      <title>Calculating Compound Interest</title>
      <style type = "text/css">
         table         { width: 300px;
                         border-collapse: collapse;
                         background-color: lightblue; }
         table, td, th { border: 1px solid black;
                         padding: 4px; }
         th            { text-align: left;
                         color: white;
                         background-color: darkblue; }
         tr.oddrow     { background-color: white; }
      </style>
```

Fig. 8.6 | Compound interest calculation with a for loop. (Part 1 of 4.)

```
<script>

   var amount; // current amount of money
   var principal = 1000.00; // principal amount
   var rate = 0.05; // interest rate

   document.writeln("<table>" ); // begin the table
   document.writeln(
      "<caption>Calculating Compound Interest</caption>" );
   document.writeln(
      "<thead><tr><th>Year</th>" ); // year column heading
   document.writeln(
      "<th>Amount on deposit</th>" ); // amount column heading
   document.writeln( "</tr></thead><tbody>" );

```

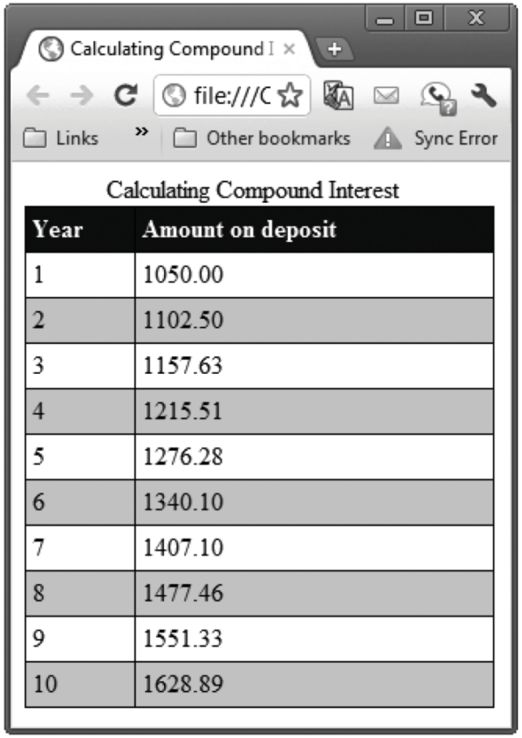
Fig. 8.6 | Compound interest calculation with a `for` loop. (Part 2 of 4.)

```
      // output a table row for each year
      for ( var year = 1; year <= 10; ++year )
      {
        amount = principal * Math.pow( 1.0 + rate, year );

        if ( year % 2 !== 0 )
          document.writeln( "<tr class='oddrow'><td>" + year +
            "</td><td>" + amount.toFixed(2) + "</td></tr>" );
        else
          document.writeln( "<tr><td>" + year +
            "</td><td>" + amount.toFixed(2) + "</td></tr>" );
      } //end for

      document.writeln( "</tbody></table>" );

    </script>
  </head><body></body>
</html>
```

Fig. 8.6 | Compound interest calculation with a for loop. (Part 3 of 4.)

為何*amount.toFixed(2)*可以運作？

Auto-box: https://developer.mozilla.org/en-US/docs/Glossary/Primitive



Year	Amount on deposit
1	1050.00
2	1102.50
3	1157.63
4	1215.51
5	1276.28
6	1340.10
7	1407.10
8	1477.46
9	1551.33
10	1628.89

Fig. 8.6 | Compound interest calculation with a `for` loop. (Part 4 of 4.)

8.4 Examples Using the for Statement (cont.)

- JavaScript includes an exponentiation operator ****** (introduced in ES2016).
  - The expression x ** y calculates the value of x raised to the power of y.
- For compatibility, the *Math.pow(x, y)* method can also be used now.

8.7 break and continue Statements

- break statement in a while, for, do…while or switch statement
  - Causes *immediate exit* from the statement
  - Execution continues with the next statement in sequence
- break statement common uses
  - Escape early from a loop
  - Skip the remainder of a switch statement

8.7 break and continue Statements (Cont.)

- `continue` statement in a `while`, `for` or `do…while`
  - skips the remaining statements in the body of the statement and proceeds with the next iteration of the loop
  - In `while` and `do…while` statements, the loop-continuation test evaluates immediately after the continue statement executes
  - In `for` statements, the increment expression executes, then the loop-continuation test evaluates

```
<!DOCTYPE html>

<!-- Fig. 8.11: BreakTest.html -->
<!-- Using the break statement in a for statement. -->
<html>
   <head>
      <meta charset = "utf-8">
      <title>
         Using the break Statement in a for Statement
      </title>
      <script>

         for ( var count = 1; count <= 10; ++count )
         {
            if ( count == 5 )
               break; // break loop only if count == 5

            document.writeln( count + " " );
         } //end for

```

Fig. 8.11 | Using the break statement in a for statement. (Part 1 of 2.)

```
         document.writeln(
            "<p>Broke out of loop at count = " + count + "</p>" );

      </script>
   </head><body></body>
</html>
```

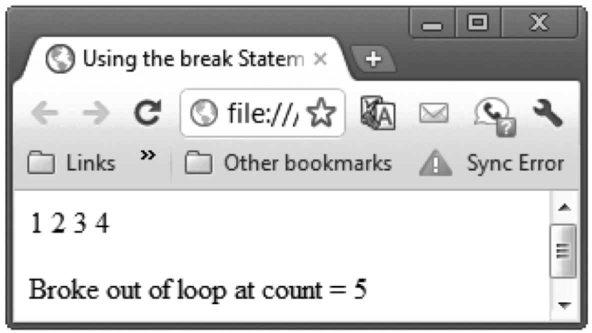


Fig. 8.11 | Using the `break` statement in a `for` statement. (Part 2 of 2.)

```
<!DOCTYPE html>

<!-- Fig. 8.12: ContinueTest.html -->
<!-- Using the continue statement in a for statement. -->
<html>
   <head>
      <meta charset = "utf-8">
      <title>
         Using the continue Statement in a for Statement
      </title>

      <script>

         for ( var count = 1; count <= 10; ++count )
         {
            if ( count == 5 )
               continue; // skip remaining loop code only if count == 5

            document.writeln( count + " " );
         } //end for

```

Fig. 8.12 | Using the continue statement in a for statement. (Part 1 of 2.)

```
            document.writeln( "<p>Used continue to skip printing 5</p>" );

         </script>

      </head><body></body>
   </html>
```

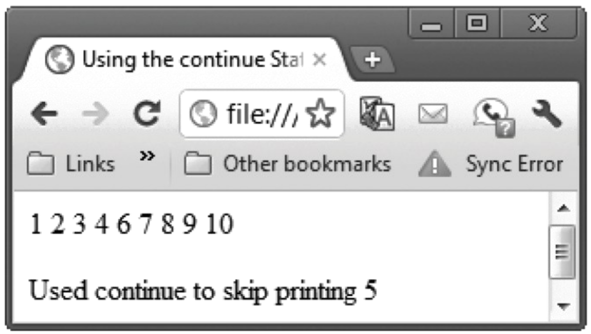


Fig. 8.12 | Using the `continue` statement in a `for` statement. (Part 2 of 2.)

8.8 Logical Operators

- Logical operators can be used to form complex conditions by combining simple conditions
  - && (logical AND)
  - || (logical OR)
  - ! (logical NOT, also called logical negation)
- The && operator is used to ensure that two conditions are both true before choosing a certain path of execution
- JavaScript evaluates to `false` or `true` all expressions that include relational operators, equality operators and/or logical operators

expression1	expression2	expression1 && expression2
false	false	false
false	true	false
true	false	false
true	true	true

Fig. 8.13 | Truth table for the && (logical AND) operator.

8.8 Logical Operators (Cont.)

- The || (logical OR) operator is used to ensure that either or both of two conditions are true before choosing choose a certain path of execution

expression1	expression2	expression1 || expression2
false	false	false
false	true	true
true	false	true
true	true	true

Fig. 8.14 | Truth table for the || (logical OR) operator.

8.8 Logical Operators (Cont.)

- The && operator has a higher precedence than the || operator
- Both operators associate from left to right.
- An expression containing && or || operators is evaluated only until truth or falsity is known
  - This is called short-circuit evaluation

8.8 Logical Operators (Cont.)

- ! (logical negation) operator
  - reverses the meaning of a condition (i.e., a true value becomes false, and a false value becomes true)
  - Has only a single condition as an operand (i.e., it is a unary operator)
  - Placed before a condition to evaluate to `true` if the original condition (without the logical negation operator) is `false`

Conversion from Non-boolean values to Boolean Values

- Most nonboolean values can be converted to a boolean `true` or `false` value
  - Nonzero numeric values are considered to be `true`
  - The numeric value zero is considered to be `false`
  - Any string that contains characters is considered to be `true`
  - The empty string is considered to be `false`
  - The value `null` and variables that have been declared but not initialized are considered to be `false`
  - All objects are considered to be `true`

可以讓變數直接作為if的condition！

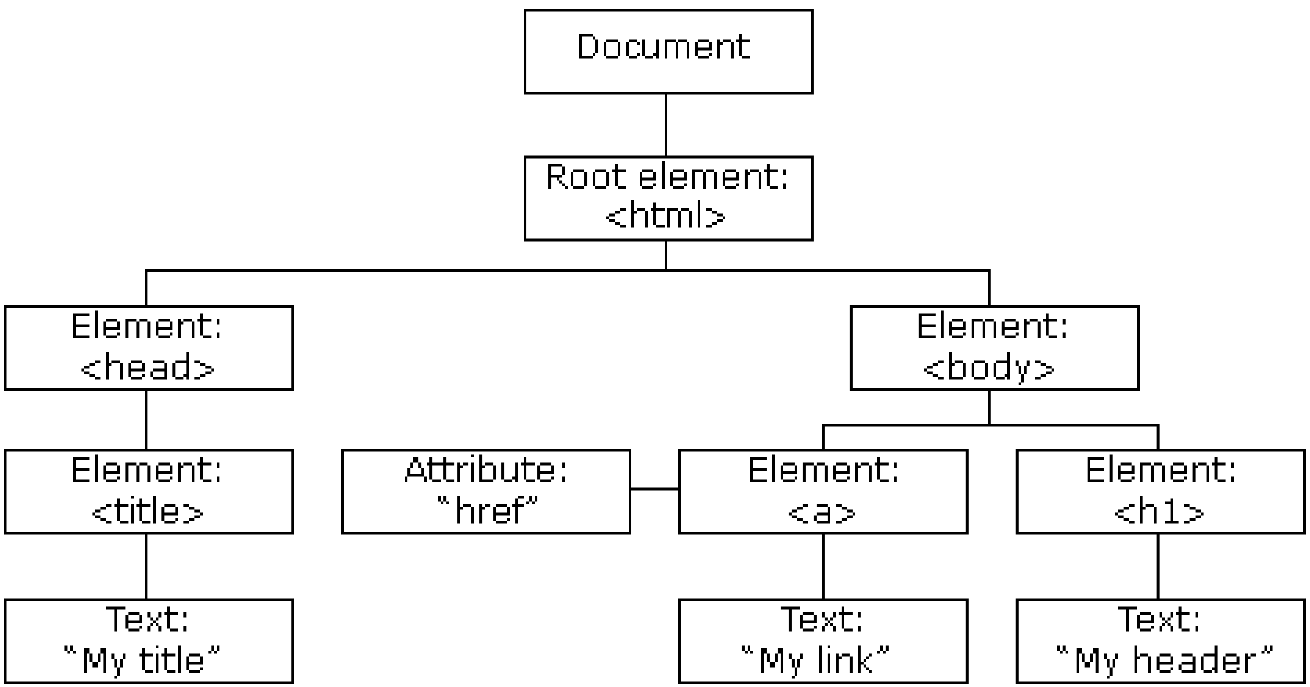
Logical operators

Logical operators		
Operator	Usage	Description
Logical AND (&&)	expr1 && expr2	Returns expr1 if it can be converted to false; otherwise, returns expr2. Thus, when used with Boolean values, && returns true if both operands are true; otherwise, returns false.
Logical OR (||)	expr1 || expr2	Returns expr1 if it can be converted to true; otherwise, returns expr2. Thus, when used with Boolean values, || returns true if either operand is true; if both are false, returns false.
Logical NOT (!)	!expr	Returns false if its single operand that can be converted to true; otherwise, returns true.

Please check supplementary example: js-03-logical-operators.html

DOM (Document Object Model)1

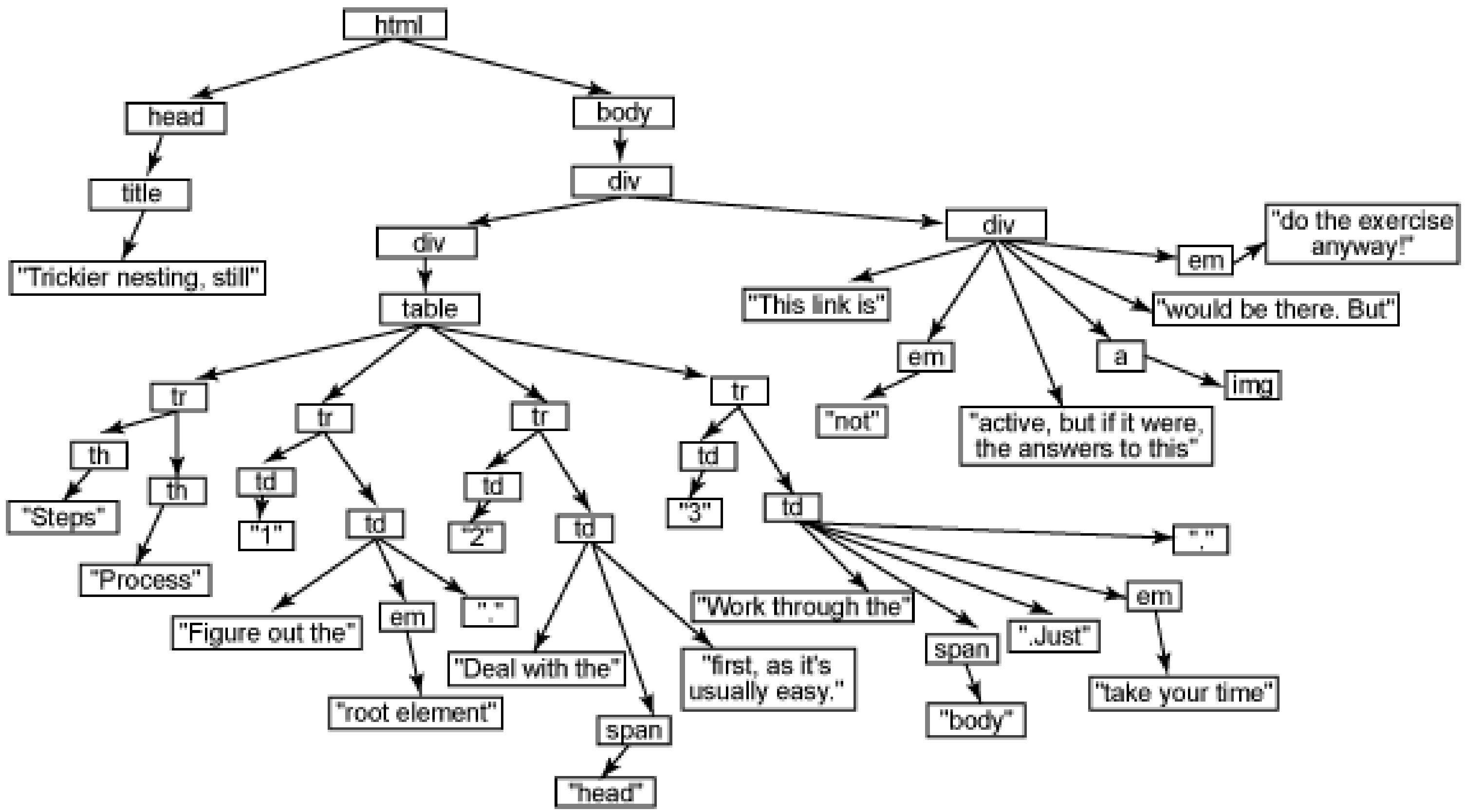
- When a web page is loaded, the browser creates a Document Object Model of the page.
- The HTML DOM model is constructed as a tree of Objects. (樹狀結構的一堆物件)



http://www.w3schools.com/js/js_htmldom.asp

DOM (Document Object Model)2

- A DOM Example for a Page



https://vinaytech.files.wordpress.com/2008/11/domimage.png

在HTML元素上動態顯示訊息1

- 我們若要只改動整張網頁的一個小區塊，我們先要取得那個區塊的元素(element)的物件(object)
  - 這個物件就是文件物件(DOM object)
  - 要能取得某元素的物件，這個物件要先幫他取個id
    - id: 身分識別(identification)，類似身份證字號的概念，是不重複的、唯一的名稱。
    - id只要跟一般變數名稱一樣的風格即可，重點是不要重複。
  - 要用**document.getElementById(id)**取得物件
  - 設定的區塊通常是<div>元素或<p>元素
    - 用**objectName.innerHTML = "…"** 去動態設定它的內容(請看後續範例)。

在HTML元素上動態顯示訊息2

- 我們可以在元素內**動態**塞入純文字

```
<p id="demo">Hello World!</p>

<script>
    let x = document.getElementById("demo");
    x.innerHTML = "Hello DOM!";
</script>
```

Hello DOM!

js-04-simple-dom.html

在HTML元素上動態顯示訊息3

- 也可以在元素內**動態塞入HTML片段** [js-05-dynamic-dom.html]

Cute Baby

```
<h2>寶寶圖片選擇器</h2>
<div id="demo"></div>
<script>
let choice = window.prompt("請選擇(1)男孩(2)女孩:");
let imageUrl = '';
if (choice == 1) {
    imageUrl = 'http://cdn2.momjunction.com/wp-content/uploads/2015/02/Hispanic-Baby-Names-For-Your-Little-Boy.jpg';
} else if (choice == 2) {
    imageUrl = 'http://cdn2.momjunction.com/wp-content/uploads/2015/04/Top-188-Latest-And-Modern-Hindu-Baby-Girl-Names.jpg';
} else {
    alert("無效的選擇!");
}
if (imageUrl) {
    document.getElementById("demo").innerHTML =
    `<img src="${imageUrl}" alt="寶寶圖片">`;
```

Cute Baby

ES6: Template Literals (Strings)

- Template literals are literals delimited with backtick (`) characters, allowing for multi-line strings, string interpolation with embedded expressions.
  - https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Template_literals
- Example: https://www.w3schools.com/js/tryit.asp?filename=tryjs_templates_variables

```
let firstName = "John";
let lastName = "Doe";

let text = `Welcome ${firstName}, ${lastName}!`;
```